SOFTWARE DESIGN DOCUMENT (SDD)

Kelompok 7 - EduScale

31 Juli 2026

Contents

SOFTWARE DESIGN DOCUMENT (SDD)

EduScale – Sistem Manajemen Sekolah Berbasis Web

Nama Proyek: EduScale School Management System
Versi Dokumen: 1.0
Tanggal: 31 Juli 2026
Kelompok: 7
Mata Kuliah: Scalable System Design – Final Project

---

1. Pendahuluan

1.1 Tujuan Dokumen

Dokumen ini mendeskripsikan desain teknis sistem EduScale, mencakup arsitektur, struktur database, desain API, dan desain komponen frontend. Dokumen ini menjadi acuan teknis pengembangan dan pengujian.

1.2 Lingkup

Dokumen ini mencakup seluruh subsistem EduScale: - Backend REST API (Node.js/Express) - Database Schema (PostgreSQL/Sequelize) - Frontend Single Page Application (React/Vite)

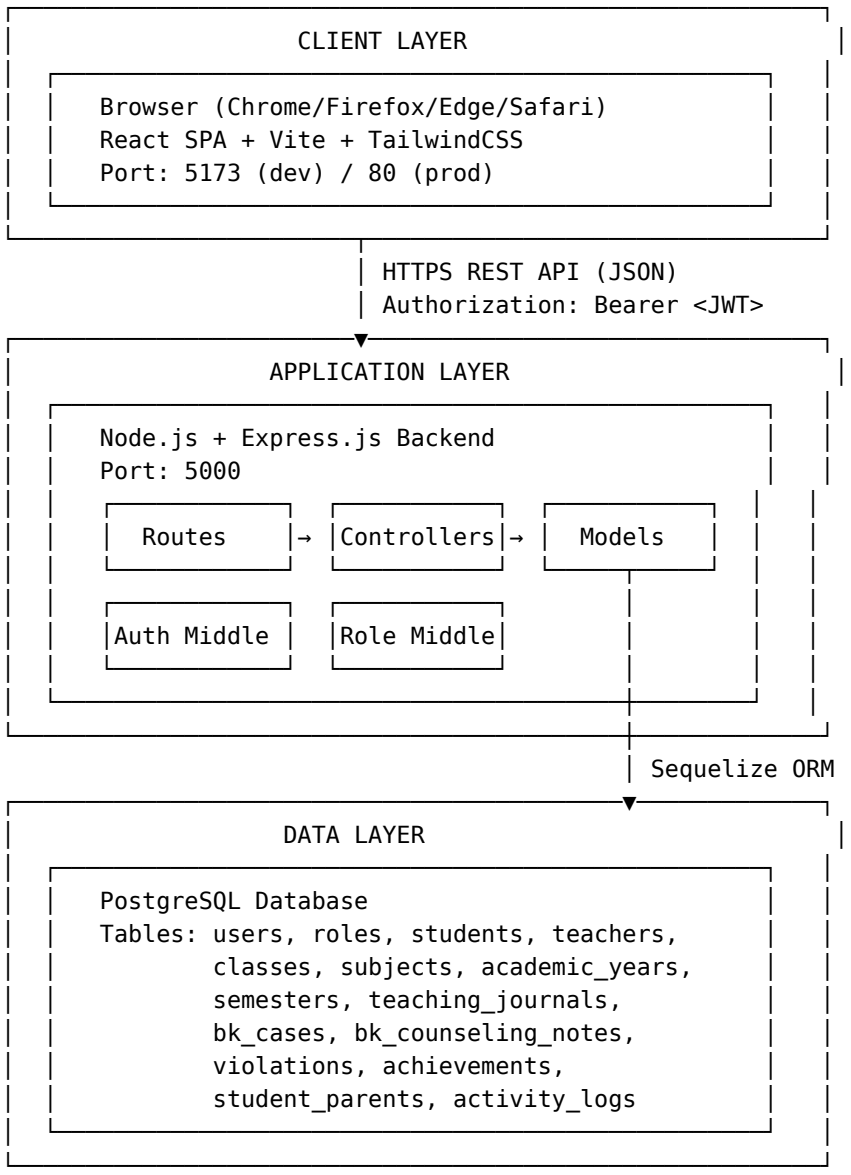
1.3 Referensi

- SRS EduScale v1.0
- IEEE Std 1016-2009: Software Design Descriptions

---

2. Arsitektur Sistem

2.1 Gambaran Arsitektur (High-Level)



2.2 Pola Arsitektur

Backend: MVC (Model-View-Controller) - **Model:** Definisi tabel dan relasi menggunakan Sequelize ORM - **Controller:** Logika bisnis dan penanganan request/response - **Route:** Pendefinisian endpoint dan middleware chain

Frontend: Component-Based Architecture - **Pages:** Komponen halaman utama - **Layout:** Komponen tata letak bersama (AdminLayout, Navbar, Sidebar) - **Context:** State management global (AuthContext) - **Routes:** Konfigurasi routing (React Router v7)

2.3 Teknologi Stack

Layer	Teknologi	Versi
Frontend Framework	React.js	^19.2.7
Build Tool	Vite	^8.1.1
CSS Framework	TailwindCSS	^4.3.2
HTTP Client	Axios	^1.18.1
Routing	React Router DOM	^7.18.1
Charts	Chart.js + react-chartjs-2	^4.5.1
Backend Framework	Express.js	^5.2.1
ORM	Sequelize	^6.37.8
Database	PostgreSQL	≥13
Auth	JWT (jsonwebtoken)	^9.0.3
Password Hash	bcrypt	^6.0.0
Runtime	Node.js	≥18

---

3. Desain Database

3.1 Entity Relationship Diagram (Deskripsi Tekstual)

Entitas Utama dan Relasi:

```
roles ──────────────── users
  (1)               (N)    role_id FK

users ──────────────── teachers
  (1)               (0..1)  user_id FK

academic_years ──────── semesters
       (1)              (N)    academic_year_id FK

academic_years ──────── classes
       (1)              (N)    academic_year_id FK

teachers ──────────────── classes (wali_kelas)
  (1)                  (N)    wali_kelas_id FK

classes ──────────────── students
  (1)                  (N)    class_id FK

teachers ──────────────── subjects
  (1)                  (N)    teacher_id FK
```

```
teachers ──────────── teaching_journals
  (1)                 (N)    teacher_id FK

classes ──────────── teaching_journals
  (1)                 (N)    class_id FK

subjects ──────────── teaching_journals
  (1)                 (N)    subject_id FK

semesters ──────────── teaching_journals
  (1)                 (N)    semester_id FK

students ──────────── bk_cases
  (1)                 (N)    student_id FK

bk_cases ──────────── bk_counseling_notes
  (1)                 (N)    bk_case_id FK

students ──────────── violations
  (1)                 (N)    student_id FK

students ──────────── achievements
  (1)                 (N)    student_id FK

students ──────────── student_parents
  (1)                 (N)    student_id FK

users ──────────── activity_logs
  (1)                 (N)    user_id FK
```

3.2 Skema Tabel

Tabel: `roles`

Kolom	Tipe	Constraint
id	INTEGER	PK, AUTO_INCREMENT
name	STRING	NOT NULL
createdAt	DATETIME	AUTO
updatedAt	DATETIME	AUTO

Tabel: `users`

Kolom	Tipe	Constraint
id	INTEGER	PK, AUTO_INCREMENT
name	STRING	NOT NULL
email	STRING	NOT NULL, UNIQUE
password	STRING	NOT NULL (hashed)
role_id	INTEGER	FK ⟶ roles.id
createdAt	DATETIME	AUTO
updatedAt	DATETIME	AUTO

Tabel: `students`

Kolom	Tipe	Constraint
id	INTEGER	PK, AUTO_INCREMENT
nis	STRING	UNIQUE
name	STRING	NOT NULL
gender	STRING	-
birth_place	STRING	-
birth_date	DATEONLY	-
address	TEXT	-
phone	STRING	-
parent_name	STRING	-
status	STRING	(Aktif/Lulus/Pindah/Keluar)
class_id	INTEGER	FK ⟶ classes.id
createdAt	DATETIME	AUTO
updatedAt	DATETIME	AUTO

Tabel: `teachers`

Kolom	Tipe	Constraint
id	INTEGER	PK, AUTO_INCREMENT
nip	STRING	UNIQUE
name	STRING	NOT NULL
gender	STRING	-
phone	STRING	-
address	TEXT	-
specialization	STRING	-
status	STRING	-
user_id	INTEGER	FK ⟶ users.id
createdAt	DATETIME	AUTO
updatedAt	DATETIME	AUTO

Tabel: `classes`

Kolom	Tipe	Constraint
id	INTEGER	PK, AUTO_INCREMENT
name	STRING	NOT NULL
grade	STRING	-
academic_year_id	INTEGER	FK ⟶ academic_years.id
wali_kelas_id	INTEGER	FK ⟶ teachers.id
createdAt	DATETIME	AUTO
updatedAt	DATETIME	AUTO

Tabel: `subjects`

Kolom	Tipe	Constraint
id	INTEGER	PK, AUTO_INCREMENT
name	STRING	NOT NULL
code	STRING	-
teacher_id	INTEGER	FK ⟶ teachers.id
createdAt	DATETIME	AUTO
updatedAt	DATETIME	AUTO

Tabel: `academic_years`

Kolom	Tipe	Constraint
id	INTEGER	PK, AUTO_INCREMENT
year	STRING	NOT NULL
is_active	BOOLEAN	DEFAULT false
createdAt	DATETIME	AUTO
updatedAt	DATETIME	AUTO

Tabel: `semesters`

Kolom	Tipe	Constraint
id	INTEGER	PK, AUTO_INCREMENT
name	STRING	NOT NULL
academic_year_id	INTEGER	FK ⟶ academic_years.id
createdAt	DATETIME	AUTO
updatedAt	DATETIME	AUTO

Tabel: `teaching_journals`

Kolom	Tipe	Constraint
id	INTEGER	PK, AUTO_INCREMENT
teacher_id	INTEGER	FK ⟶ teachers.id
class_id	INTEGER	FK ⟶ classes.id
subject_id	INTEGER	FK ⟶ subjects.id
semester_id	INTEGER	FK ⟶ semesters.id
date	DATEONLY	NOT NULL
topic	STRING	NOT NULL
description	TEXT	-
attendance_count	INTEGER	-
createdAt	DATETIME	AUTO
updatedAt	DATETIME	AUTO

Tabel: `bk_cases`

Kolom	Tipe	Constraint
id	INTEGER	PK, AUTO_INCREMENT
student_id	INTEGER	FK → students.id
type	STRING	-
description	TEXT	-
date	DATEONLY	-
status	STRING	(Proses/Selesai)
createdAt	DATETIME	AUTO
updatedAt	DATETIME	AUTO

Tabel: `bk_counseling_notes`

Kolom	Tipe	Constraint
id	INTEGER	PK, AUTO_INCREMENT
bk_case_id	INTEGER	FK → bk_cases.id
note	TEXT	-
date	DATEONLY	-
createdAt	DATETIME	AUTO
updatedAt	DATETIME	AUTO

Tabel: `violations`

Kolom	Tipe	Constraint
id	INTEGER	PK, AUTO_INCREMENT
student_id	INTEGER	FK → students.id
type	STRING	-
description	TEXT	-
date	DATEONLY	-
points	INTEGER	-
createdAt	DATETIME	AUTO
updatedAt	DATETIME	AUTO

Tabel: `achievements`

Kolom	Tipe	Constraint
id	INTEGER	PK, AUTO_INCREMENT
student_id	INTEGER	FK → students.id
type	STRING	-
title	STRING	-
level	STRING	-
date	DATEONLY	-
createdAt	DATETIME	AUTO
updatedAt	DATETIME	AUTO

Tabel: `activity_logs`

Kolom	Tipe	Constraint
id	INTEGER	PK, AUTO_INCREMENT
user_id	INTEGER	FK → users.id
action	STRING	NOT NULL
description	TEXT	-
createdAt	DATETIME	AUTO
updatedAt	DATETIME	AUTO

---

4. Desain API (REST Endpoints)

4.1 Konvensi API

- **Base URL:** `http://localhost:5000/api`
- **Format Data:** JSON
- **Autentikasi:** `Authorization: Bearer <JWT_TOKEN>`
- **HTTP Status Code:**
  - `200 OK` – Sukses baca/update
  - `201 Created` – Sukses tambah data
  - `400 Bad Request` – Input tidak valid
  - `401 Unauthorized` – Token tidak ada/tidak valid
  - `403 Forbidden` – Tidak memiliki izin role
  - `404 Not Found` – Data tidak ditemukan
  - `500 Internal Server Error` – Error server

4.2 Endpoint Autentikasi

Method	Endpoint	Auth	Deskripsi
POST	`/auth/register`	No	Registrasi pengguna baru
POST	`/auth/login`	No	Login, dapatkan JWT
GET	`/auth/profile`	Yes	Lihat profil pengguna aktif

Request Body – POST /auth/login:

```
{
  "email": "admin@eduscale.id",
  "password": "admin123"
}
```

Response – POST /auth/login (200 OK):

```
{
  "message": "Login berhasil",
  "token": "eyJhbGciOiJIUzI1NiIs...",
  "user": {
    "id": 1,
    "name": "Administrator",
    "email": "admin@eduscale.id",
    "role": { "name": "Admin" }
  }
}
```

4.3 Endpoint Siswa

Method	Endpoint	Auth	Role	Deskripsi
GET	`/students`	Yes	Semua	Daftar siswa (search, filter, paginasi)
GET	`/students/:id`	Yes	Semua	Detail siswa
POST	`/students`	Yes	Admin	Tambah siswa
PUT	`/students/:id`	Yes	Admin	Update siswa
PUT	`/students/:id/status`	Yes	Admin	Update status siswa
DELETE	`/students/:id`	Yes	Admin	Hapus siswa
GET	`/students/export`	Yes	Admin	Export CSV siswa
POST	`/students/import`	Yes	Admin	Import siswa (bulk)

Query Parameters – GET /students: - `search` – filter nama/NIS - `class_id` – filter kelas - `status` – filter status - `page` – halaman (default: 1) - `limit` – jumlah per halaman (default: 20)

4.4 Endpoint Guru

Method	Endpoint	Auth	Role	Deskripsi
GET	`/teachers`	Yes	Semua	Daftar guru
GET	`/teachers/:id`	Yes	Semua	Detail guru
POST	`/teachers`	Yes	Admin	Tambah guru
PUT	`/teachers/:id`	Yes	Admin	Update guru
DELETE	`/teachers/:id`	Yes	Admin	Hapus guru

4.5 Endpoint Kelas

Method	Endpoint	Auth	Role	Deskripsi
GET	`/classes`	Yes	Semua	Daftar kelas
GET	`/classes/:id`	Yes	Semua	Detail kelas
POST	`/classes`	Yes	Admin	Tambah kelas
PUT	`/classes/:id`	Yes	Admin	Update kelas
DELETE	`/classes/:id`	Yes	Admin	Hapus kelas

4.6 Endpoint Mata Pelajaran

Method	Endpoint	Auth	Role	Deskripsi
GET	`/subjects`	Yes	Semua	Daftar mapel
GET	`/subjects/:id`	Yes	Semua	Detail mapel
POST	`/subjects`	Yes	Admin	Tambah mapel
PUT	`/subjects/:id`	Yes	Admin	Update mapel
DELETE	`/subjects/:id`	Yes	Admin	Hapus mapel

4.7 Endpoint Jurnal Mengajar

Method	Endpoint	Auth	Role	Deskripsi
GET	`/journals`	Yes	Semua	Daftar jurnal
GET	`/journals/:id`	Yes	Semua	Detail jurnal
POST	`/journals`	Yes	Admin, Guru	Tambah jurnal
PUT	`/journals/:id`	Yes	Admin, Guru	Update jurnal
DELETE	`/journals/:id`	Yes	Admin	Hapus jurnal

4.8 Endpoint BK

Method	Endpoint	Auth	Deskripsi
GET	`/bk/cases`	Yes	Daftar kasus BK
POST	`/bk/cases`	Yes	Tambah kasus BK
PUT	`/bk/cases/:id`	Yes	Update kasus BK
DELETE	`/bk/cases/:id`	Yes	Hapus kasus BK
GET	`/bk/violations`	Yes	Daftar pelanggaran
POST	`/bk/violations`	Yes	Tambah pelanggaran
DELETE	`/bk/violations/:id`	Yes	Hapus pelanggaran
GET	`/bk/achievements`	Yes	Daftar prestasi
POST	`/bk/achievements`	Yes	Tambah prestasi
DELETE	`/bk/achievements/:id`	Yes	Hapus prestasi

4.9 Endpoint Dashboard

Method	Endpoint	Auth	Deskripsi
GET	`/dashboard/stats`	Yes	Statistik utama
GET	`/dashboard/activities`	Yes	Aktivitas terbaru
GET	`/dashboard/charts`	Yes	Data grafik

4.10 Endpoint Pengguna

Method	Endpoint	Auth	Role	Deskripsi
GET	`/users`	Yes	Admin	Daftar pengguna
GET	`/users/:id`	Yes	Admin	Detail pengguna
POST	`/users`	Yes	Admin	Tambah pengguna
PUT	`/users/:id`	Yes	Admin	Update pengguna
DELETE	`/users/:id`	Yes	Admin	Hapus pengguna
PUT	`/users/:id/change-password`	Yes	Semua	Ganti password

4.11 Endpoint Tahun Ajaran & Audit Log

Method	Endpoint	Auth	Deskripsi
GET	`/academic-years`	Yes	Daftar tahun ajaran
POST	`/academic-years`	Yes	Tambah tahun ajaran
GET	`/activities`	Yes	Audit log aktivitas
GET	`/roles`	Yes	Daftar role

---

5. Desain Komponen Frontend

5.1 Struktur Direktori Frontend

```
eduscale-frontend/src/
├── api.js                    # Axios instance + interceptors
├── App.jsx                   # Root component
├── main.jsx                  # Entry point
├── context/
│   └── AuthContext.jsx       # Global auth state + token management
├── layout/
│   ├── AdminLayout.jsx       # Layout wrapper dengan sidebar + navbar
│   ├── Navbar.jsx            # Navbar atas
│   └── Sidebar.jsx           # Navigasi sidebar kiri
├── pages/
│   ├── Login.jsx             # Halaman login
│   ├── Dashboard.jsx         # Halaman dashboard + grafik
│   ├── Students.jsx          # Manajemen siswa
│   ├── Teachers.jsx          # Manajemen guru
│   ├── Classes.jsx           # Manajemen kelas
│   ├── TeachingJournal.jsx   # Jurnal mengajar
│   ├── BKCases.jsx           # Bimbingan konseling
│   ├── Users.jsx             # Manajemen pengguna
│   ├── AuditLog.jsx          # Log aktivitas
│   ├── Settings.jsx          # Pengaturan
│   └── ChangePassword.jsx    # Ganti password
└── routes/
    ├── AppRoutes.jsx         # Definisi routing aplikasi
    └── ProtectedRoute.jsx    # Guard route untuk halaman yang memerlukan
autentikasi
```

5.2 Desain AuthContext

```
// Context menyediakan:
{
  user: { id, name, email, role },   // Data user aktif
  token: "JWT_TOKEN",                // Token tersimpan di localStorage
  login(email, password),            // Fungsi login
  logout(),                          // Fungsi logout + redirect
```

```
  isAuthenticated: true/false        // Status autentikasi
}
```

5.3 Desain Axios Instance (api.js)

```
// Konfigurasi:
- baseURL: import.meta.env.VITE_API_URL || "http://localhost:5000/api"
- Request Interceptor: Auto-attach Bearer token dari localStorage
- Response Interceptor: Handle 401 → logout otomatis
```

5.4 Alur Routing

```
/login                 → Login.jsx (public)
/                      → Dashboard.jsx (protected)
/students              → Students.jsx (protected)
/teachers              → Teachers.jsx (protected)
/classes               → Classes.jsx (protected)
/subjects              → Subjects.jsx (protected)
/journals              → TeachingJournal.jsx (protected)
/bk                    → BKCases.jsx (protected)
/users                 → Users.jsx (protected, admin)
/audit-log             → AuditLog.jsx (protected, admin)
/change-password       → ChangePassword.jsx (protected)
```

5.5 Desain State Management per Halaman

Setiap halaman data (Students, Teachers, dll.) menggunakan pola:

```
State:
- data: []            # Array data
- loading: boolean    # Loading state
- search: string      # Query pencarian
- page: number        # Halaman aktif
- totalPages: number  # Total halaman
- showModal: boolean  # Tampilkan modal form
- editItem: null      # Item sedang diedit

Lifecycle:
- useEffect → loadData() saat mount dan saat filter berubah
- CRUD → panggil api → reload data → tutup modal
```

---

6. Desain Keamanan

6.1 Autentikasi – JWT Flow

```
Client                    Server
  |                         |
  |── POST /auth/login ────►|
  |   {email, password}     | 1. Cari user berdasarkan email
  |                         | 2. bcrypt.compare(password, hash)
  |                         | 3. jwt.sign({id, email, role_id})
```

```
  |◄── 200 OK {token} ──────|
  |                         |
  |── GET /api/resource ──►|
  |   Bearer: <token>        | 4. jwt.verify(token, JWT_SECRET)
  |                         | 5. req.user = decoded payload
  |◄── 200 OK {data} ──────|
```

6.2 Authorization – RBAC Flow

```
// Middleware Chain untuk endpoint Admin:
router.post("/students",
  authMiddleware,          // Verifikasi JWT
  authorizeRoles("Admin"), // Verifikasi role
  studentController.create
);

// roleMiddleware.js:
// Cek req.user.role_id → cocokkan dengan nama role
// Jika tidak cocok → 403 Forbidden
```

6.3 Perlindungan Input

- Body size limit: `express.json({ limit: "10mb" })` – mencegah payload flooding
- Password hashing: `bcrypt.hash(password, 10)` – melindungi data password
- SQL Injection: Sequelize ORM menggunakan parameterized queries secara default
- XSS: React secara default melakukan escaping pada output JSX

---

7. Desain Skalabilitas

7.1 Paginasi Database

Semua endpoint list menggunakan paginasi dengan `LIMIT` dan `OFFSET`:

```
const offset = (page - 1) * limit;
await Model.findAndCountAll({ limit, offset });
```

7.2 Query Optimization

- Hanya kolom yang diperlukan yang di-select (`attributes: [...]`)
- Include (JOIN) hanya digunakan saat benar-benar diperlukan
- Pencarian menggunakan `Op.iLike` dengan wildcard (dapat ditingkatkan dengan full-text index)

7.3 Stateless Architecture

- Backend stateless (tidak ada server-side session)
- JWT memungkinkan load balancing horizontal tanpa sticky session
- Database dapat dipisahkan ke server terpisah (read replicas)

7.4 Potensi Peningkatan Skalabilitas (Rekomendasi)

Komponen	Kondisi Saat Ini	Rekomendasi Production
Auth	JWT stateless	Tambah refresh token
Caching	Tidak ada	Redis untuk data statis
Rate Limiting	Tidak ada	express-rate-limit
CORS	`cors()` terbuka	Batasi origin production
Load Balancer	Tidak ada	Nginx reverse proxy
Database Pool	Default Sequelize	Konfigurasi pool size

---

8. Alur Data (Data Flow)

8.1 Alur Login

```
1. User → Form Login → AuthContext.login()
2. AuthContext → POST /auth/login → Backend
3. Backend → Validasi email + password (bcrypt)
4. Backend → Generate JWT token
5. Backend → Response 200 {token, user}
6. AuthContext → Simpan token ke localStorage
7. AuthContext → Set user state
8. React Router → Redirect ke Dashboard
```

8.2 Alur CRUD Data (Contoh: Tambah Siswa)

```
1. Admin → Klik "Tambah Siswa" → showModal = true
2. Admin → Isi form → Submit
3. Students.jsx → api.post("/students", formData)
4. Axios Interceptor → Tambah Authorization header
5. Backend → authMiddleware → Verifikasi JWT
6. Backend → roleMiddleware → Verifikasi "Admin"
7. Backend → studentController.create() → Student.create()
8. Backend → logActivity() → ActivityLog.create()
9. Backend → Response 201 {message, student}
10. Students.jsx → Reload daftar siswa
11. Modal ditutup, notifikasi sukses ditampilkan
```

8.3 Alur Audit Log

```
Setiap operasi CREATE/UPDATE/DELETE:
  → Controller memanggil logActivity(user_id, action, description)
  → utils/logActivity.js → ActivityLog.create()
  → Data tersimpan di tabel activity_logs
  → Dashboard/AuditLog dapat membaca log
```

Dokumen ini merupakan acuan teknis pengujian dan pengembangan sistem EduScale.

Versi: 1.0 | **Tanggal:** 31 Juli 2026 | **Kelompok:** 7